LINFO2132 Project Statement

Project Overview

The goal of the project is to implement your own compiler for an imperative language (defined by us) from scratch in Java. You will implement the compiler in several phases (corresponding to the phases of the compiler pipeline, i.e., lexing, parsing, etc.). For each phase, we will give you the description of what we expect from you and the deadline by which you must submit your solution. Each phase will be evaluated. All phases together will count towards 60% of your final grade.

Specific instructions on how to submit on Inginious are given in this document.

You *must* use the project skeleton given on Teams to start. It is a Gradle project, and you should be able to compile it with Gradle. For example, if you use IntelliJ and are in the first phase, importing such a project will open a window “Gradle” in which you can compile your project with “gradle build”, then execute it with "gradle run --args="-lexer <input.lang>".

The first phase: The lexer

Your first job will be to implement a lexer for the programming language. You can find an example that shows most of the syntactic elements of that language in the file “code_example.lang”. Looking at the example program, you will notice the following syntactic elements:

- We only use characters corresponding to the printable characters in the ASCII standard plus the space character, tab (‘\t’ in Java), and newline (‘\n’ in Java).
- Comments start with #
- Identifiers, i.e., names of variables, functions, and types: they can contain letters and digits and underscores, but they must start with a non-capital letter or underscore, i.e., the following words are valid identifiers: abc, abc123, _abc_, _12s. Apart from the first letter, they can include capital letters.
- All collection names must begin with a capital letter.
- Keywords: final, coll, def, for, while, if, else, return, not, ARRAY
- Values for the different base types:
    - Natural numbers INT (32-bit, corresponding to int values in Java)
    - FLOAT numbers (32-bit, corresponding to float values in Java). You only need to consider the decimal point (e.g., 3.24) and you don't have to implement lexing for numbers in scientific notation (e.g. 4.5e-2). ".234" should be recognized as a float "0.234" and 00342 as an integer "324".

    - STRINGs. They are enclosed in double quotation marks and can contain any printable character, space characters, and escaped characters, i.e., special notations for some characters, namely \n (newline), \\ (backslash), and \" .
    - BOOLean values: true and false.
- A variable can be declared immutable (`final` keyword), and all variables are pre-typed ("INT a" and not "a INT").
- Various special symbols: =, +, -, *, /, %, ==, =/=, <, >, <=, >=, (,), {, }, [,], . (dot), &&, ||, ; (semicolon), and the comma.
- Symbols **might** be separated by space, newline and tabulator characters (the so-called *whitespace*). For instance, these two inputs would generate the same sequence of symbols: `INT x = 12;`, `INT x=12;`

Implementation and submission on Inginious

1. You must pass a Reader object representing the input to the Lexer constructor.
2. You must implement a method `getNextSymbol()` which returns the next symbol in the sequence of symbols lexed from the input.
3. Each returned symbol must be printable (think object-oriented: all symbols implement a method to print them). All symbols should have concise et precise names when you print them, but their exact formatting is up to you. A token <STRING, "Hello"> or <String, Hello> is equivalent to us.
4. We will run your code with the command "javac main.java -lexer filepath". The argument "-lexer" indicates that we want only the Lexer to run, and its output should be printed. (one symbol per line for readibility). The argument "filepath" will be the file used as input in your main.

There is no report to submit for this part.

Apart from that, you are free to add anything you find necessary (other classes, methods, etc.). Some general advice:

- It's up to you how you want to represent the end of the input. You could let getNextSymbol() return a special symbol or just default class that represents null.
- You can also freely decide how you want to represent symbols: as a single Java class, as multiple subclasses (for each token type), etc.
- It will be much more convenient for the parser if getNextSymbol does not return symbols for whitespaces and comments.

Error Handling

During the whole compilation process, multiple errors might arise. It is important to understand when each of these errors must be thrown out at the right time.

For instance, nothing forbids the programmer to write something like `INT a= "Hello" / 2;

This is syntactically correct, but semantically wrong. It is thus not the role of the Lexer or Parser to throw such error.

The role of the lexer is to transform the input file into a sequence of symbols. Hence the only errors you must handle are unrecognised tokens (e.g., a "@" character). Once a syntactic error is recognised, the Lexer should throw an error and your main should return something else than 0.

What you must not do

We expect from you that you write the lexer by hand. It is forbidden to

- use lexer or parser generation tools or the code generated by them,
- use lexer and parsing tools and libraries (external or in the JDK, such as the Scanner, Pattern and Matcher classes),
- copy solutions (or parts of them) from your fellow students or from the Internet.

The second phase: The parser

The goal of this phase is to produce an Abstract Syntax Tree (AST) from the output of the Lexer. You must transform the sequence of symbols produced by the Lexer into a tree structure that represents your program. You must respect the following guidelines:

- The AST must respect the syntax of the language.
- You should implement recursive-descent parsing. Detailed example is in slides 5b; slides 5c which show how operator associativity might also be useful. You will have to hand-tune the recursive parser: "If parts of the grammar are not LL(1), you can write special parser code for them." (slides 5b).
- If the sequence of symbols does not respect it, then you must report an error. For example, `a int 2` is syntactically incorrect (the `=` is missing).
- You must not report semantic errors (e.g. `INT a = "Hello"`)
- Implement the parser in a class Parser with the Lexer as constructor parameter. You should have a method `getAST()` that returns the root of the AST.

Implementation and submission on Inginious

1. Each AST node must be printable. All nodes should have concise et precise names.
2. We will run your code with the command "javac main.java -parser filepath". The argument "-parser" indicates that we want up to the Parser to run, and its output should be printed in the form of a tree, as indicated below. The argument "filepath" will be the file used as input in your main. For example, the printed output of "INT x = 1 + 2" would be: (note that variations can exist with your nodes; for example, "ArithmeticOperator, +" might be "AddOperator"; the important is that the role of each node is understandable)

Expr

    Type, INT

    Identifier, x

    AssignmentOperator

    Expr

        Integer, 1

        ArithmeticOperator, +

        Integer, 2

Some tips:

- You might see a design problem/bug in your Lexer when implementing the AST generation. Do not hesitate to refactor your code. This is true for the entire project, and you should not be afraid to change some previous parts to facilitate the overall coding of your compiler.
- You'll need to code a lot of boilerplate code. Try to use abstract classes, interface, etc. as efficiently as possible.
- You should re-use the test cases defined for the Lexer. This will ease the test part. Remember, one different test file is expected for all four phases of the project.
- You should also create new test cases based on what is syntactically (in)correct.
- If you implement the `equals(Object o)` method in your objects, you can use the assert methods from JUnit with your custom objects (e.g., `assertEquals(expectedRoot, myRoot)`)
- The parser can handle operation precedence.
- At this phase of the project, you might already throw some semantic errors, which is good, but we'll verify and grade the errors you find only at the third phase.

Report

You should report the full grammar rules of your language. Note that it can be modified or expanded upon (in case it's slightly incomplete, or for extra features) later in the project without penalty, but we expect to see the rules for all basic functions.

In case your parser does not handle specific cases, you should specify it in your report. Later in the project, you can complete them for a better grade. Do not forget to specify in later reports when you improved your grammar rules or parser, otherwise we won't see it.

Third phase: Semantic analysis

At the end of the previous phase, you should have produced, from the input file, a syntactically correct AST. Hence, you know that the input file respects the syntax of the language. But this does not tell you if all the operations you do in the file are correct from the semantic point of view. For instance, the syntax of the language does not prevent you from assigning an integer to a float variable. Such errors are called “semantic errors” because they are linked to the semantic of the language.

In this phase you must analyse your AST and report any semantic errors, which are mainly type errors. Our language is strongly typed, and we do static type checking, i.e., most of the type checking happens during the compilation. For type equivalence, you can use named equivalence (easy) or structural equivalence (more difficult).

You should at least handle the following cases. Each case must lead to a program crash with a custom error message with the main function returning a value different from 0 and not null (e.g. “System.exit(2)”). It does not matter when (during parsing, semantic analysis, ...) in your code the error is detected and thrown. The important thing is that the error is thrown with a message containing one of the specific keywords if appropriate or a custom one in the case of a non-listed error.

We assign a keyword to each of the following cases, which must appear in your error message.

- The type of variable, value and constant are correct. The right-hand side (rhs) of an assignment should have the same type as declared in the left-hand side (lhs); keyword: “TypeError”
- Collection declarations are correctly registered as new types for variables and cannot overwrite existing types (e.g. “coll INT”, “coll while”, or overwrite a previously defined structure). The name of collections must begin with a capital letter. Keyword of the error: “CollectionError”
- Binary operators operate on the correct types: same numeric type for arithmetic operations, boolean for comparison operators. Same for unary operators. Keyword: “OperatorError”
- Function calls have the correct parameter types (including collection constructor). Keyword: “ArgumentError”
- Loop and if constructs have boolean conditions. This error is also thrown if there is the condition is not a boolean. Keyword: “MissingConditionError”
- Return statements return the correct type in functions. Keyword: “ReturnError”
- The scopes are handled correctly. This means that when an identifier is used you should check that it is in the current scope, or a parent scope. Shadowing of global variables by function parameters is allowed. An error can arise if a variable is called before being defined, or more generally if a variable is used out of scope. Keyword: “ScopeError”

Your implementation of the Semantic Analysis must respect these two points:

1. Your program should throw an error that includes the associated keyword for each of the above cases. The program must return a value other than 0 and not null in these cases.
2. You must create at least one test for each of the points above and check if the correct error is thrown. We will check and run these tests. If need be, you can write a short report to show how to run them, but it should be as simple as possible.

Some tips:

- If the first part was done correctly, a traversal of the AST should be able to handle most of the points above. However, it might happen that the design of your AST is not optimal (which is normal!). If you find that the structure of your AST is not well suited for the semantic analysis part, we advise you to refactor it. If you do not manage to handle all these points by the deadline, try to still submit something as it will always positively impact your final grade and refactor after, for the next phase, if necessary.
- Remember that after the semantic analysis, you'll have to generate code based on the AST. A good structure is thus essential.
- You're also free to generate more errors as you wish, try to be creative! A good compiler is not only able to correctly generate code based on its input but must also give a clear explanation of why the compilation failed. And it should point out the errors in the source file. These errors must be checked by additional tests, or we will not see/grade them.

There is no report in this phase.

Fourth phase: Code generation

In this final phase, you will generate code using your AST tree. At this phase, you should also add extra features to your language.

All technical information and resources are detailed in slides 10a and 10b (you will be able to find them in the Teams channel "General", under the files tab). For the submission and grading:

1. The code will be partly evaluated on Inginious, like in the previous phases. We will use gradle and the following arguments when running your project: "source_file -o target_file". Your compiler should generate code based on the program in *source_file* and generate it in a class file whose path is *target_file*. This last argument is optional (hence the *-o*) and should be generated in a default class file of your choice if not indicated.
2. Important remark: your compiler and implementation will partly be evaluated through your own tests, so their quality should be a focus. We *will* look at your tests and grade them.
3. The statement mentions an "extra features" part. It is mainly evaluated through your report and associated tests. All extra features in the language should be illustrated in

the report (program examples) and tested in your test files. Possible extra features include more complex array programming, higher-order programming, error messages that identify more precisely faults, new built-in operations that are not included in the basic requirements, warning messages when memory is badly managed, building functions inside functions, code optimisation, extra polish on existing features that enhances them in some way, syntactic sugar for new boolean operations ... We naturally do not expect all of that.

Some tips to pass Inginious:

- (IMPORTANT TO PASS INGINIOUS) The input code has a function "def main()", which is the main function of test.class. You should put its contents in the "public static void main" method. We encourage you to test on your machine to compile the following code:

```
def main() {
    println("hello");
}
```

  You should be able to run the .class generated by hand and it should write “hello” on the standard output.
- If you are given the arguments "./tests/script.lang -o ./tests/test.class", it should save the test.class in the given argument ("./tests/test.class").
- You should save all collections .class files in the same folder as the main .class file, *not* in the root folder of your project.
- Please test if a function that returns nothing works with your compiler. Forgetting this case is a very common mistake.
- Initialising two variables with the same name, once in the main function, and another in an auxiliary function, can cause the ambiguous error "java.lang.VerifyError: Control flow falls through code end" if not handled correctly.

Report

The report contains:

- A concise explanation of the current progress of your compiler.
- The implementation choices you made, if they have significant impact on performance, for example.
- The updated grammar rules of your language.
- If you improved one the previous phases that was incomplete, you should mention what.
- The extra features of your

- A link to your github repository (do not forget to add bastien.wiaux@uclouvain.be and pierre.martou@uclouvain.be on the repo)
- A clear overview of which AI tools were used (if any) and what they were used for. Be precise.
- The quality of your report (clarity, conciseness, organisation, quality of writing, respect for the instructions) will be part of the grading. Be as direct as possible: no flowery language, and an image is worth a thousand words.

Submissions and grading

All submissions will be done on Inginious. For some submissions, we expect a zip file containing your current code, and a report (two separate tasks).

- The code contains:
    - Source files of your current progress.
    - Test files for each part of your project (for the current deadline; e.g. only one test file at the first deadline for the Lexer). These test files should be well-organised and easy to run, as we can use them to check your progress, and they WILL be graded.

The grading and expected deliverables are as follows. Exact percentages are subject to changes.

- Lexer 15%: code
- Parser 20%: code and report
- Semantic analysis 20%: code and report
- Code generation 30%: code and report
- Extra features: 15%

Notice that 15% is reserved for extra features, graded at the same time as the final phase, Code Generation. We recommend to focus on extra features only when you are done with all previous phases.

You are free to enhance your compiler in any way you find interesting. This can be with new language features, optimisation in the code generation, less restriction in the source file etc. All enhancements can be regarded as extra features.

If you are not sure whether some of your ideas are doable, do not hesitate to contact the teaching assistants.

Resources

You can find on Moodle:

- A file with all the language's features.
- A skeleton of gradle project with the signature for the Lexer (you must keep the project as a gradle project)

Deadline

- Lexer: Tuesday March 3, 23:59
- Parser: Tuesday March 24, 23:59
- Semantic analysis: Tuesday April 14, 23:59
- Code generation: Friday May 15, 23:59